# פרויקט בקורס מערכות הדמיה ברפואה

**The project code can be found at https://github.com/1danmi/imaging-systems-project**

**And you can test your images at https://colab.research.google.com/github/1danmi/imaging-systems-project/blob/master/main.ipynb**

## Preparing the Data

I started the project with a small chest X-ray collection, spanning three target classes. The raw images arrived in every shape and size, often with different exposure levels, so my first task was to standardize them.

I converted each image to grayscale to make it easier to work with. Since I needed to resize all the images to be the same size to fit in the model, I picked 224x224 pixel, as it seems to preserve enough detail to recognize anatomical structures, fit easily in GPU memory, and most importantly, matched the input size expected by most ImageNet pretrained networks that I wanted to use later.

The biggest problem with this dataset though, was its size. The dataset contains only around 250 images, which are also not distributed evenly among the classes. When training a model on such small dataset, it tends to overfit and not generalize well enough for unseen images.

To combat overfitting, I leaned heavily on data augmentation. I wrote a small processor that performs the following data augmentation techniques:

- Horizontal flip
- Rotation by a small random degree
- Translation (moving along the axes) by a small random amount of pixels
- Brightness and contrast jitter
- Adding random noise
- Contrast Limited Adaptive Histogram Equalization – a interesting technique that enhance the local contrast when needed (implemented by OpenCV)

By applying these transformations on the fly during training, I effectively multiplied the dataset and exposed the models to a wider variety of plausible X-ray appearances.

## Choosing a Model

I began modelling with a straightforward convolutional network. My baseline comprised four convolution–ReLU–max-pool blocks for feature extraction, followed by global average pooling and two fully connected layers with dropout. I chose this layout because it was easy to reason about and quick to train, giving me a reliable starting point for debugging the rest of the pipeline.

With that baseline running, I moved on to **transfer learning**. I loaded a ResNet-18 pretrained on ImageNet, froze all its convolutional layers, and replaced the final classifier with a single trainable fully connected layer tailored to the three target classes. Training only that head allowed me to benefit from the rich visual features learned from millions of images without overfitting the small X-ray dataset.

While surveying related work I noticed that some researchers apply **Capsule Networks (CapsNet)** to radiographs. Capsules output vectors that encode both the presence and pose of features, and dynamic

routing lets them agree on higher level structures—properties that sounded promising for X-ray interpretation. I implemented a small CapsNet with primary capsules feeding into class-level capsules so I could compare this architecture against the CNN approaches.

## Training the Model

I wrote a single training loop that handled batching, mixed-precision training, logging, and early stopping for all architectures. Every model used cross-entropy loss with the Adam optimiser (batch size 16, up to 30 epochs), and I scheduled the learning rate with ReduceLROnPlateau to gently nudge it lower when the validation loss stalled. During the first few runs I accidentally placed the augmentation pipeline **before** splitting the data; the same image could appear in both training and test sets with different transformations, and the models reported a misleading 100 % accuracy. After discovering the issue I corrected the order—split first, augment second—which produced realistic metrics.

## Evaluating the Model

For evaluation I relied on **5-fold cross validation**. In each fold I trained from scratch, cycling through different augmentation subsets to see which combinations helped or hurt. I logged the results of every experiment to CSV files in the results directory so I could compare them later. The best SimpleCNN variant, using the full augmentation suite, averaged about **94.8 %** accuracy. ResNet-18 with transfer learning pushed the average to roughly **96.5 %**. The CapsNet was more temperamental: with flips, rotations, and translations it reached around **94.8 %**, but certain augmentation mixes caused notable drops, reminding me that capsules can be sensitive to input perturbations.

## Parameter Tuning

I ran out of time before I could start an extensive hyperparameter search. Learning rates, weight decay, and model depths stayed at their initial values, and I did not experiment with alternative optimisers or schedulers. Most of the tuning energy instead went into comparing augmentation permutations and verifying that the 224×224 resolution was the best trade-off.

## Conclusions

This project let me build an end-to-end pipeline for classifying chest X-rays. By carefully preparing and augmenting the data, trying both conventional and capsule-based architectures, and evaluating with cross validation, I achieved realistic accuracy numbers while uncovering a few pitfalls along the way. ResNet-18 with a frozen backbone emerged as the top performer, but I found the CapsNet experiment enlightening and worth revisiting with more tuning and data. Future work will focus on hyperparameter searches and scaling the dataset to further improve the models.

**The project code can be found at https://github.com/1danmi/imaging-systems-project**

**And you can test your images at https://colab.research.google.com/github/1danmi/imaging-systems-project/blob/master/main.ipynb**